VELLORE INSTITUTE OF TECHNOLOGY

SCHOOL OF COMPUTER SCIENCE ENGINEERING AND INFORMATION SYSTEMS



A Case Study

on

Designing a Technically Coherent NFVI Architecture for Educational Evaluation

(Network Functions Virtualisation Infrastructure)

Submitted By

Aniket verma

23BIT0014

SCORE

Under the Guidance of

Prof. Suresh V J

SCORE

VIT

1. ABSTRACT

This case study designs a technically coherent Network Functions Virtualisation Infrastructure (NFVI) for educational evaluation. The proposed laboratory separates physical compute, storage and networking resources from their virtual abstractions, the infrastructure manager and the network functions deployed above them. A VM-based baseline uses Linux KVM, Open vSwitch and OpenStack to host repeatable tenant network services. One controller and three compute hosts provide a feasible teaching platform; the controller is explicitly a single point of failure, so the laboratory is not presented as a highly available production cloud. Management, tenant, external and storage traffic have distinct logical domains. A worked capacity model, tenant isolation controls, placement procedure, service path, fault scenarios and evaluation protocol make the architecture testable. Numerical values are design assumptions and calculated allocations, not measured performance. The study demonstrates how to evaluate resource feasibility, correct forwarding, isolation and recovery while preserving a clear boundary between an educational demonstrator and a carrier-grade NFV platform.

2. INTRODUCTION

Network Functions Virtualisation replaces selected dedicated network appliances with software functions running on general-purpose infrastructure. NFVI is the infrastructure foundation: physical compute, storage and network resources together with the virtualisation layer that exposes virtual resources to workloads. The ETSI NFV architectural framework distinguishes NFVI, virtual network functions and management and orchestration responsibilities [1]. This separation is central to the design: a firewall VM is a workload, not the infrastructure manager.

An educational NFVI should be understandable, reproducible and safe to experiment with. Students should be able to create isolated networks, instantiate functions, observe traffic, exhaust a declared quota and remove a compute node without losing the distinction between a planned capability and an observed result. A technically coherent design must explain which component allocates CPU, which establishes connectivity, where persistent data resides and which failures the laboratory can tolerate.

The baseline uses virtual machines rather than combining multiple orchestration systems in one unexplained stack. OpenStack manages infrastructure resources, KVM provides VM execution and Open vSwitch participates in virtual networking. Containers can be evaluated later as a separate extension. OpenStack is not assumed to provide a complete ETSI NFV Orchestrator or VNF Manager simply because it provisions VMs.

The report proposes one controller and three compute hosts connected to a managed switch. A small tenant service places a firewall and a router in series between a client network and a protected server network. An observation VM receives only explicitly mirrored traffic. The laboratory evaluates forwarding and tenant boundaries rather than claiming 5G radio performance, production availability or universal line-rate packet processing.

3. PROBLEM STATEMENT

Design an NFVI laboratory that can host multiple isolated VM-based network services, expose measurable resource limits and support controlled failure experiments. The architecture must map each requirement to an actual infrastructure component and define how correctness will be checked. It must remain feasible for an institutional laboratory without describing a single-controller installation as a fault-tolerant production platform.

Requirement	Design response	Evaluation evidence
Compute abstraction	KVM/libvirt on three dedicated compute hosts; Nova and Placement manage allocations.	VM inventory, host assignments and resource counters.
Network isolation	OpenStack projects, Neutron networks, isolated overlays and scoped security groups.	Cross-tenant negative connectivity tests.
Service forwarding	Two-interface firewall and router VMs form the mandatory client-to-server path.	Packet captures and firewall counters in the required order.
Resource feasibility	Explicit host reserve, no baseline CPU overcommit and tenant quotas.	Capacity worksheet and rejected oversized requests.
Recoverability	Recreate workloads from approved images and backed-up configuration.	Observed restoration time and explicit state-loss record.
Repeatability	Versioned images, descriptors and configuration automation.	Recreated topology matches the declared manifest.

4. MOTIVATION OF THE CASE STUDY

An NFV diagram can look complete while omitting basic operational dependencies. For example, a virtual link cannot isolate tenants if its identifiers are reused incorrectly; a second VM does not create high availability if both copies share the same host; and a snapshot does not replace tested backup and restoration. This study makes those dependencies explicit.

• **Learning motivation:** relate hardware capacity, hypervisor scheduling, virtual networks and network-function behavior through visible laboratory experiments.

• **Engineering motivation:** use a small, internally consistent stack with documented ownership of compute, networking, images and identity.

• **Evaluation motivation:** define pass conditions and collect evidence for isolation, forwarding, resource accounting and controlled recovery.

The expected benefit is clearer assessment and repeatable experimentation. A monetary saving or throughput improvement is not claimed without measurements from the actual laboratory.

5. OBJECTIVES

1. Design a technically coherent NFVI architecture for educational evaluation.
2. Identify physical, virtualisation, virtual-resource, management and workload boundaries.
3. Specify feasible hardware capacity and separate management, tenant, external and storage traffic.
4. Deploy repeatable tenant network services with explicit resource and isolation controls.
5. Define placement, configuration, monitoring, rollback and cleanup procedures.
6. Create a quantitative capacity example and a controlled evaluation matrix.
7. Distinguish calculated design feasibility from unmeasured performance and availability.

6. METHODOLOGY

The method begins with a conventional ETSI NFV reference model, then selects one implementable educational stack. A component map assigns every architectural function to software or hardware. A capacity budget reserves infrastructure overhead before admitting workloads. Network paths and trust boundaries are specified before service deployment. Finally, acceptance tests define how each requirement would be observed in a controlled laboratory.

Stage	Activity	Output
Model	Separate infrastructure, workloads and orchestration.	Layered architecture and responsibility table.
Size	Reserve host overhead and calculate normal/failure capacity.	Resource budget and admission limits.
Connect	Define segments, gateways and firewall interfaces.	Address plan and explicit forwarding path.
Control	Define tenant roles, quotas and ownership.	Access policy and cleanup procedure.
Evaluate	Specify load, fault and denial tests.	Evidence checklist and result recording template.

This is an architecture and evaluation design. The capacity worksheet is calculated from declared assumptions; no OpenStack cluster or production network was deployed for this report. Evaluation outcomes remain to be measured, and the proposed test matrix does not imply that those tests have already passed.

6.1 Proposed Framework / Architecture

The baseline consists of one controller C0, three compute hosts W1-W3, a managed Ethernet switch and an external laboratory gateway. The controller runs management services; worker hosts execute tenant VMs. All four machines require hardware virtualisation support where appropriate, consistent time synchronisation and controlled administrative access.

6.1 Proposed Framework / Architecture

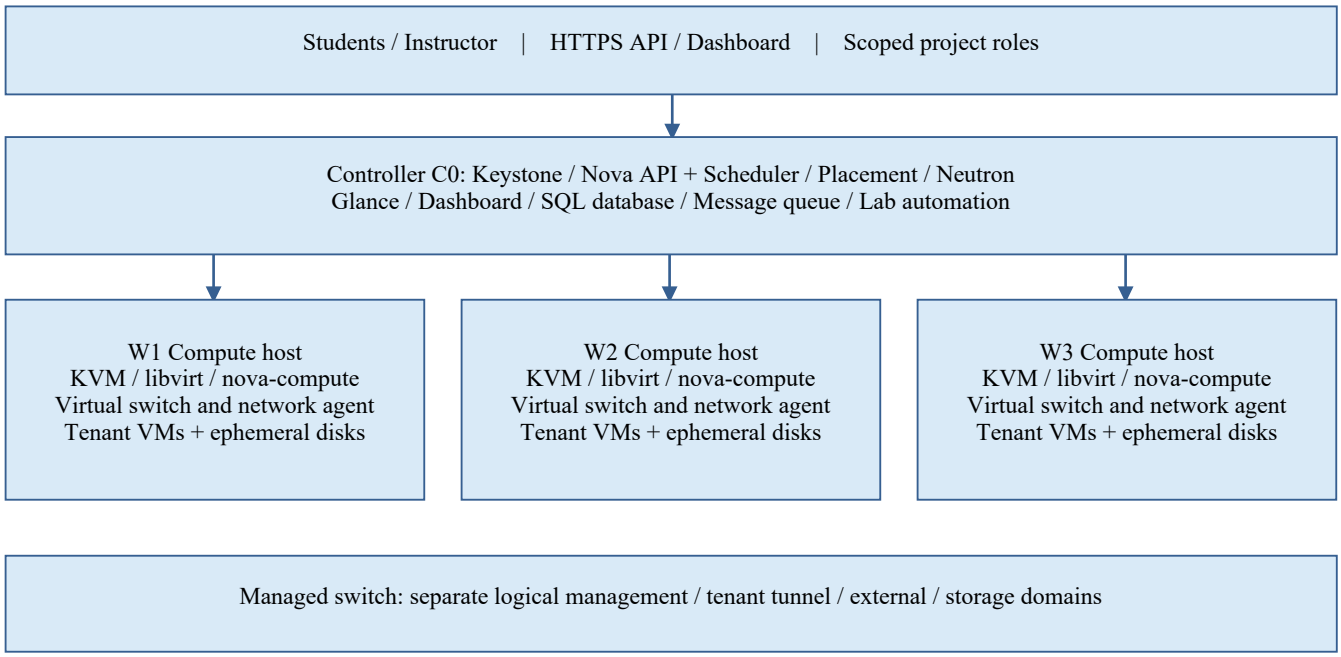


Fig. 1. Educational NFVI topology. C0 is intentionally a single controller and is not highly available.

Layer	Selected implementation	Boundary
Physical resources	Three compute servers, controller, SSDs, Ethernet NICs and managed switch.	Real capacity and shared failure dependencies.
Virtualisation	KVM, libvirt and virtual switching.	VM execution and virtual interface attachment.
Virtual resources	vCPU, guest RAM, ephemeral disk and Neutron ports/networks.	Allocated resources visible to tenant workloads.
Infrastructure management	OpenStack Nova/Placement, Neutron, Glance and Keystone.	Infrastructure provisioning and API-level authorization.
Network functions	Linux router/firewall and optional observation VMs.	Service behavior and application state.
Service lifecycle	Instructor-controlled descriptors and automation.	Limited educational workflow; not a full ETSI NFVO/VNFM.

Nova and Placement provide compute resource management, while Neutron provides networking. Glance distributes approved images and Keystone supplies identity and role-based API access [2]. The baseline uses ephemeral VM disks; Cinder is optional and excluded from the capacity arithmetic. Adding Cinder requires an explicitly sized and protected storage backend rather than assuming that a service name supplies redundant storage.

OpenStack control services also depend on their database and message queue. C0 and the switch are shared dependencies. Losing C0 can interrupt management operations while existing guest forwarding may continue; the actual behavior must be measured for the selected deployment and network agents.

6.1.1 Network and Storage Design

Domain	Example addressing / mechanism	Access rule
Management	VLAN 10; 10.10.10.0/24; C0 and W1-W3 management interfaces.	Only administrator jump host and required service peers.
Tenant transport	VLAN 20; 10.10.20.0/24 tunnel endpoints; VXLAN overlays.	No direct student access to tunnel or host interfaces.
External laboratory	VLAN 30; 192.0.2.0/24 documentation-only example.	Controlled gateway; no unintended route to campus networks.
Storage / backup	VLAN 40; 10.10.40.0/24 when a backup target is provided.	Administrator and backup services only.
Tenant A networks	10.101.1.0/24 client; 10.101.2.0/24 protected.	Only Tenant A project ports and declared inter-network route.
Tenant B networks	10.102.1.0/24 client; 10.102.2.0/24 protected.	No permitted path from Tenant A without an explicit exception.

The external 192.0.2.0/24 range is illustrative and must be replaced by an approved laboratory allocation. VLANs are logical separation on the same switch, not independent physical redundancy. A single physical NIC may carry tagged domains for a low-cost lab, but separate NICs improve administrative separation and reduce interference. Neither arrangement alone proves throughput isolation.

VXLAN introduces encapsulation overhead. Configure a guest MTU compatible with the entire underlay and verify it using nonfragmenting probes; do not assume a 1500-byte guest packet fits a 1500-byte tunnel underlay. Overlay identifiers must be unique within their intended scope, and the selected Neutron backend must be configured consistently [3].

Ephemeral disks contain disposable guest images and temporary workload data. Store descriptors, firewall rules and image checksums outside the tenant VMs, then back them up to an independent target. A copy on C0 alone does not protect against loss of C0. Stateful firewall sessions are not preserved merely by backing up configuration.

6.1.2 Explicit Service Forwarding

The evaluated path is client -> firewall -> router -> protected server [6]. For Tenant A, the firewall links client subnet 10.101.1.0/24 to transit subnet 10.101.254.0/30; the router links that transit subnet to protected subnet 10.101.2.0/24. The client uses the firewall as its gateway, and the server uses the router. Forward and return routes traverse both functions. A direct bridge or alternate router between the subnets would bypass the evaluated function and must be absent. Security groups complement this topology; they do not automatically enforce service-chain order.

An observation VM is attached to a separate monitor network and receives an explicitly configured mirror when supported. It is not automatically an inline forwarding hop. Interface forwarding and any exceptions to Neutron anti-spoofing or port-security rules must be narrowly scoped to the approved router/firewall ports; tenant-wide disabling is not required.

6.2 Working Process

1. **Prepare:** verify hardware virtualisation, management access, time synchronisation, underlay connectivity and approved addressing.

2. **Install:** configure one consistent supported OpenStack release, KVM workers and the selected Neutron networking backend. Pin configuration and image versions.

3. **Register:** create instructor and student roles, projects, approved images, flavors and quotas. Keep administration credentials outside student guests.

4. **Describe:** record the service interfaces, image IDs, CPU/RAM/disk requests, tenant ID, security rules and permitted host separation.

5. **Admit:** validate project quota, residual capacity and replica placement before creating resources. Reject incomplete or oversized requests.

6. **Provision:** create tenant networks and ports, instantiate VMs, apply routes and firewall policy, then verify guest readiness.

7. **Validate:** test the allowed path, a blocked flow, a bypass attempt and cross-project denial. Collect counters and captures.

8. **Evaluate:** apply controlled traffic and failure scenarios, record measurements, then restore the declared baseline.

9. **Clean up:** delete only resources tagged to the experiment, release quotas and confirm no residual ports, disks or credentials remain.

6.3 Resource Admission and Placement Algorithm

Use an explainable capacity-first policy. The baseline applies a 1:1 vCPU-to-available-physical-core budget for teaching clarity; an actual hypervisor scheduler does not guarantee a dedicated physical core without additional pinning policy. RAM and disk admission use explicit allocations. Anti-affinity places each tenant service replica on a different worker.

```
ADMIT(service, inventory, project_quota)
  validate identity, descriptor, units and image approval
  calculate complete CPU, RAM and disk demand
  reject if project quota or global budget is exceeded
  reserve a private copy of current inventory
  for each service replica:
    filter hosts by capacity and replica anti-affinity
    rank by remaining resource headroom, then host ID
    reserve all functions of this replica on chosen host
    reject and discard reservations if no host fits
  validate intended links, routes and policy
  return complete plan for controlled provisioning
```

For R replicas and H hosts, a straightforward per-replica candidate scan and sort costs O(RH log H), excluding API operations and network validation. This simple heuristic can reject placements that another packing might admit; it is not an optimal bin-packing solver. Concurrent admissions need a consistent reservation mechanism rather than independent stale snapshots.

6.4 Implementation Approach

Component	Implementation choice	Reason
Compute	Linux + KVM/libvirt; Nova compute agent.	VM lifecycle with visible host-level resource control.
Infrastructure control	OpenStack controller APIs, scheduler and Placement.	Separates infrastructure allocation from guest services.
Virtual networking	Neutron with a supported Open vSwitch-based backend.	Project networks, ports and explicit routed topology.
Images	Minimal approved Linux images in Glance.	Repeatable functions with known software provenance.
Guest functions	Linux routing and firewall rules; optional traffic tools.	Inspectable forwarding without vendor-specific telecom software.
Automation	Versioned descriptors and configuration scripts.	Repeatability, ownership tracking and safe teardown.
Observability	Host/VM metrics, virtual-switch counters and packet captures.	Correlates infrastructure load and service behavior.
Backup	Descriptors and configuration copied to an independent target.	Restoration evidence rather than an untested backup claim.

Security and Management Controls

Assign each student group a separate project and limited API role. Apply project quotas to VM instances, vCPUs, RAM, networks and ports. Restrict metadata and management endpoints as appropriate. Restrict guest SSH to authorized sources, protect image integrity and review any privileged networking exception. A shared host remains a shared security boundary and is not equivalent to a physically separate laboratory. Consult release-specific security guidance; reference [4] is an older conceptual guide, not a current configuration checklist.

Use host firewalls and switch policies to protect management networks, scoped security groups for allowed guest flows and guest firewall policy for the service being evaluated. These controls operate at different points; one cannot silently replace the others. Collect security evidence through negative tests as well as successful pings.

Failure and Rollback Model

A failed provisioning operation should remove only experiment-owned resources in reverse dependency order. If a worker fails, rebuild the affected disposable VMs from approved images on a surviving worker after capacity admission. Reapply configuration and validate the path before declaring the service restored. This is restart/recreation, not transparent live migration or stateful session preservation.

Application replicas require explicit traffic selection, health checks and state handling. Host anti-affinity alone does not implement automatic failover. The baseline assesses whether enough surviving capacity exists; it leaves actual restoration time and flow interruption as laboratory measurements.

7. ENVIRONMENTAL SETUP

7.1 Hardware Requirements

Item	Proposed laboratory specification	Accounting
Controller C0	8 CPU cores, 32 GiB RAM, 240 GiB SSD, management NIC.	Separate control capacity; not counted for tenant VNFs.
Workers W1-W3	Each: 16 physical CPU cores, 64 GiB RAM, 480 GiB SSD.	Three equal-capacity compute nodes.
Per-worker host reserve	4 cores, 16 GiB RAM and 80 GiB disk.	Leaves 12 cores, 48 GiB RAM, 400 GiB disk per worker.
Networking	Managed VLAN-capable switch; 1 GbE minimum for teaching, faster NICs optional.	Link capacity is evaluated separately from CPU capacity.
Backup target	Independent machine or approved protected storage.	Required to claim recovery after controller/storage loss.
Student access	Browser/SSH clients through controlled access.	No unrestricted campus or internet attack traffic.

7.2 Software Requirements

Software	Purpose
Supported Linux distribution	Consistent base operating system and security updates.
KVM / libvirt	Hardware-assisted VM execution and management.
One supported OpenStack release	Nova, Placement, Neutron, Keystone, Glance and required control dependencies.
Open vSwitch and compatible networking agents	Virtual switching and supported tenant overlay integration.
Minimal Linux guest images	Router/firewall, client, server and observation functions.
Traffic and diagnostic tools	iperf3, ping, packet capture and interface counters.
Configuration/version control	Repeatable deployment definitions, backups and test records.

Choose versions from the selected release compatibility guidance before installation. This report does not assert that an arbitrary combination of current operating system, OpenStack and networking versions is supported. Container infrastructure and acceleration are optional later experiments, not hidden prerequisites for the VM baseline.

7.3 Small-Laboratory Alternative

If only one powerful workstation is available, use nested virtualisation for a functional demonstration with reduced scale. Label all simulated workers as sharing one physical failure domain. Such a setup can evaluate API workflows and logical tenant separation, but cannot substantiate multi-host resilience, hardware independence or production packet-processing performance.

8. TARGET

Targets are acceptance criteria for the proposed laboratory. They are not measured results. Record actual values, test conditions and deviations when the architecture is implemented.

Criterion	Proposed acceptance condition	Evidence
Resource accounting	Every admitted workload fits declared quotas and residual CPU/RAM/disk.	Allocation table and rejected oversized request.
Allowed forwarding	Approved client traffic reaches server through the firewall.	Packet captures and matching firewall counters.
Blocked forwarding	A declared forbidden protocol or port is denied.	Connection failure plus deny log/counter.
Tenant isolation	Tenant A cannot access Tenant B private network or resources.	Negative network/API tests with timestamps.
No bypass	Client cannot reach protected server around the firewall.	Route audit and capture at all service interfaces.
Reproducibility	Recreated service matches descriptor and expected policy.	Image hashes, configuration and topology comparison.
Worker recovery	Affected VMs can be recreated within remaining capacity.	Observed interruption, rebuild and readiness times.
Controller failure	Management impact is recorded without assuming guest failure.	Separate API and existing-flow observations.

8.1 Sample Input and Output

Service descriptor	Example
Tenant	Student project A; separate identity and quota.
Functions per replica	Firewall: 2 vCPU/4 GiB RAM/10 GiB disk; router: 1/2/8; observer: 1/2/8.
Replica resource sum	4 vCPU, 8 GiB RAM, 26 GiB disk. Observer is monitored separately, not an inline chain hop.
Replica count	Two, with anti-affinity across workers.
Network intent	Client -> firewall -> router -> protected server; separate mirror/monitor network.
Expected output	Host assignments, owned networks/ports, configured routes, policy and validation checklist.

Client and protected server test generators are additional workload resources. Their budget is reserved separately in the worked example, preventing the evaluation tools from being silently omitted. The dedicated router is the protected-side next hop after the firewall; the observer receives a mirror. Returning a valid placement does not mean the network service is ready until interface configuration and packet tests succeed.

8.2 Worked Capacity and Failure-Domain Analysis

Three workers provide 3 x 12 = 36 allocatable CPU cores, 3 x 48 = 144 GiB RAM and 3 x 400 = 1,200 GiB local disk after host reserves. With one worker unavailable, nominal surviving capacity is 24 cores, 96 GiB RAM and 800 GiB local disk. These totals establish resource feasibility only; they do not prove service continuity or storage accessibility.

To retain explicit capacity for test endpoints, admit two tenant services with two replicas each. Each replica uses 4 vCPU, 8 GiB RAM and 26 GiB disk. Four replicas therefore use 16 vCPU, 32 GiB RAM and 104 GiB disk. Reserve four lightweight client/server test VMs, each with 1 vCPU, 2 GiB RAM and 8 GiB disk: another 4 vCPU, 8 GiB RAM and 32 GiB disk.

Allocation	vCPU	RAM GiB	Disk GiB
Four service replicas	16	32	104
Four test endpoint VMs	4	8	32
Total admitted evaluation load	20	40	136
Three-worker allocatable capacity	36	144	1,200
Two-worker surviving capacity	24	96	800
Headroom after one-worker loss	4	56	664

A valid normal placement is A1 and B1 on W1, A2 on W2 and B2 on W3, with the four endpoints distributed without exceeding host capacity. A simpler failure-compatible target is A1/B1 plus two endpoints on one surviving host and A2/B2 plus two endpoints on the other. Each surviving host then uses 10 vCPU, 20 GiB RAM and 68 GiB disk, below its 12/48/400 budget.

Both copies of each tenant remain on different workers after rebuilding. If an unavailable worker had the only current copy of ephemeral state, spare disk on another host cannot recover that state. The calculation supports reconstruction from image and backed-up configuration; stateful connection preservation is outside the baseline.

Network Capacity Is an Independent Constraint

A 1 GbE NIC does not imply 1 Gbit/s usable service throughput after encapsulation, packet-size effects, guest processing and shared traffic. Measure the direct baseline and the function path under identical conditions. Monitor both link directions and aggregate traffic across tenants. Set offered load below the observed loss boundary for latency experiments; report saturation experiments separately.

For each test, record packet size, direction, number of flows, offered rate, duration, VM placement, CPU allocation and background traffic. Without those conditions, a throughput number cannot be compared meaningfully with another run.

9. RESULTS AND DISCUSSION

The completed result of this study is a coherent architecture and a calculated resource budget. No physical laboratory measurements are available. The capacity arithmetic shows that the declared 20-vCPU, 40-GiB evaluation load fits the normal cluster and can be reconstructed on two surviving workers with the stated placement. Network performance, interruption time and operational isolation remain to be tested.

Experiment	Procedure	Record after implementation
Direct baseline	Send client/server traffic without the evaluated function path in an isolated comparison topology.	Goodput, RTT, loss and host/guest CPU.
Function path	Use identical load through firewall and router.	Forwarding evidence and performance difference.
Isolation	Attempt forbidden cross-project API and packet access.	Denied/succeeded, exact rule and supporting logs.
Noisy neighbor	Increase load in one tenant while observing another.	Per-tenant goodput, delay and shared NIC/CPU counters.
Capacity rejection	Request more than remaining quota or host capacity.	Rejection reason and unchanged allocations.
Worker loss	Stop one worker under instructor control; recreate affected VMs.	Outage, detection, recreation, configuration and ready times.
Controller outage	Stop management services while observing existing guest traffic.	API availability and forwarding continuity separately.
Teardown / restore	Remove owned resources, recreate from approved descriptor.	Residual-object audit and restored topology.

Run a warm-up before timing, repeat each performance test at least five times and keep both individual results and summary statistics. Use a monotonic clock for durations. Small samples justify reporting their range and median, but not strong population claims. Packet captures should use synthetic traffic and avoid collecting personal or campus network data.

9.1 Advantages

• The VM baseline maps every responsibility to an identified component and avoids treating the infrastructure manager as a full service orchestrator.

• The resource worksheet includes host overhead, replicas and test generators, making placement and failure experiments reviewable.

• The explicit routed path and negative tests support assessment of forwarding correctness and tenant isolation.

• Versioned images and descriptors support repeatable classroom exercises and controlled teardown.

9.2 Limitations

The single controller, switch and optional shared storage are common failure dependencies. The design is not production HA. Local disks are disposable, stateful failover is unimplemented, CPU isolation requires more than a simple vCPU count, and logical network isolation must be verified. The proposed results cannot establish carrier-grade availability, radio performance, universal VNF compatibility or cloud cost savings.

10. CONCLUSION

A technically coherent NFVI for educational evaluation requires more than a collection of virtual machines. The infrastructure must expose bounded compute, storage and networking resources, define who manages them and show how workload traffic traverses the intended functions. This case study supplies those elements through a conventional VM-based laboratory with explicit management, tenant and external boundaries.

The proposed one-controller, three-worker design is feasible as a teaching architecture while openly retaining its shared failure dependencies. Its resource budget includes infrastructure reserves, two tenant services with independent replicas and separate test endpoints. The admitted load fits two surviving workers by calculation, but restoration and service behavior must still be demonstrated through controlled tests.

The evaluation method separates capacity feasibility, forwarding correctness, isolation and observed performance. This allows students to explain why an experiment succeeds or fails without confusing a design target with a measured outcome. The architecture is therefore suitable as a reproducible baseline for learning NFV infrastructure management and evaluating future improvements.

11. FUTURE ENHANCEMENT

1. Introduce a multi-controller design with properly configured database and message-queue resilience; test quorum and failure behavior.

2. Add independent switch/uplink paths and explicitly validate redundancy instead of counting logical VLANs as fault domains.

3. Evaluate shared persistent storage with a declared replication, failure and backup model before hosting stateful functions.

4. Add an ETSI-aligned NFV Orchestrator and VNF Manager integration, clearly separating infrastructure and service lifecycle responsibilities.

5. Compare ordinary virtual switching with CPU pinning, huge pages, DPDK or SR-IOV in controlled experiments; account for lost flexibility and hardware requirements.

6. Add a separate container-based extension and explain its orchestration, resource, network and isolation boundaries rather than assuming Kubernetes and OpenStack are interchangeable.

7. Automate the complete experiment lifecycle and collect comparable workload traces, repeated measurements and reproducibility artifacts.

12. REFERENCES

[1] ETSI. **GS NFV 002 V1.1.1 Network Functions Virtualisation Architectural Framework.** October 2013. Foundational reference used for the conventional NFVI model.
https://www.etsi.org/deliver/etsi_gs/NFV/001_099/002/01.01.01_60/gs_NFV002v010101p.pdf

[2] OpenStack. **Nova System Architecture.** Official compute architecture and related service responsibilities.
https://docs.openstack.org/nova/latest/admin/architecture.html

[3] OpenStack. **Neutron Administration Guide.** Networking implementation and operational guidance.
https://docs.openstack.org/neutron/latest/admin/

[4] OpenStack. **Security Guide.** Conceptual security guidance; page notes it was last updated for the Train era. Check current release documentation.
https://docs.openstack.org/security-guide/

[5] ETSI. **Network Functions Virtualisation.** NFV standards overview and evolution beyond the foundational model.
https://www.etsi.org/technologies/689-network-functionsvirtualisation

[6] IETF. **RFC 7665 Service Function Chaining Architecture.** October 2015. Logical service ordering and forwarding responsibilities.
https://www.rfc-editor.org/rfc/rfc7665

Appendix A - Key Assumptions

• This is an educational architecture design; hardware, software installation and traffic measurements are not claimed as completed.

• One controller and three physically distinct workers are assumed. Nested alternatives share a physical failure domain.

• CPU admission uses a conservative 1:1 budget after host reserves; dedicated-core guarantees require additional scheduling configuration.

• VM disks are ephemeral. Rebuild uses approved images and separately backed-up configuration; stateful sessions may be lost.

• The declared capacity calculation includes four service replicas and four test endpoints. Controller and network capacity are accounted separately.

• All private/example addresses require instructor approval before use. Experiments remain isolated from unrelated campus and internet traffic.